



VICTOR QIAN

OUTILS DE TRADUCTION DE TEXTE D'IMAGE



SOMMAIRE

1. Contexte et Problématique
2. Méthodologie et Développement
3. Tests et Résultats
4. Bilan Personnel

PROBLÉMATIQUE ET CONTEXTE

- Problématique :
 - Comment traduire rapidement du texte contenu dans une image, de manière fluide et naturelle, avec une solution légère, locale et gratuite ?
- Contexte :
 - Besoin d'un outil de traduction visuelle similaire à Google Lens
 - Destiné aux utilisateurs apprenant une langue étrangère
 - Doit fonctionner localement (sans cloud), en temps quasi réel



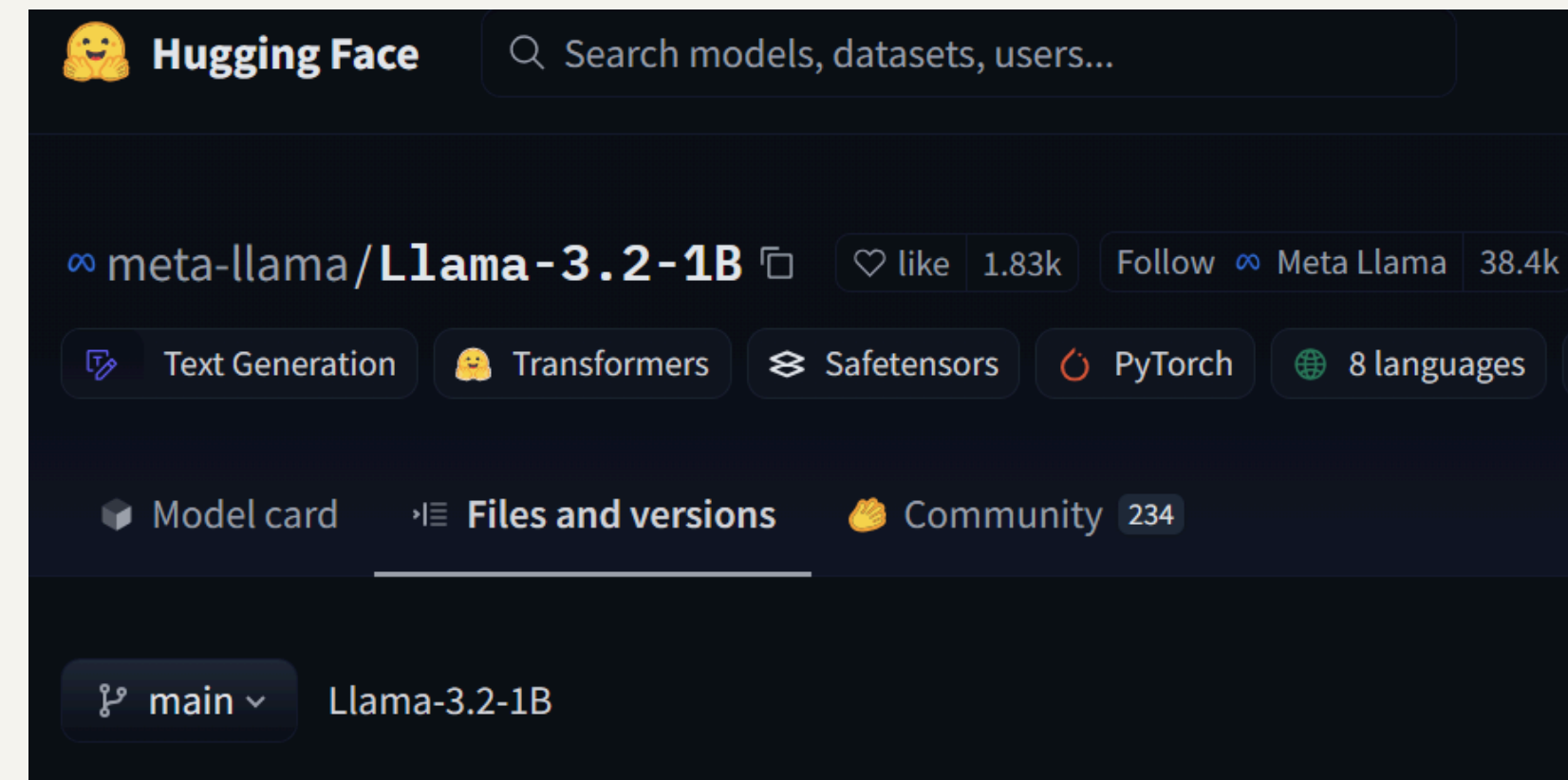
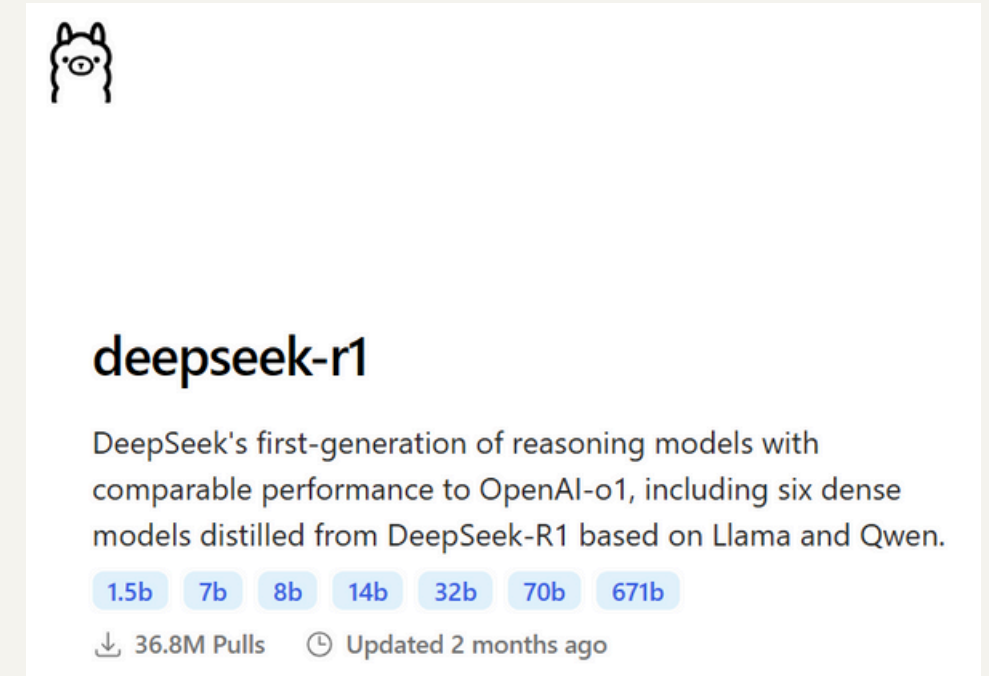
MÉTHODOLOGIE

- Recherche d'outils OCR et de modèles de traduction adaptés
- Découpage du projet en modules (OCR, UI, traduction)
- Approche itérative : tests réguliers et ajustements
 -

```
▼ core
  > __pycache__
  📄 screen_capture.py
  📄 tempCodeRunnerFil...
  📄 translator.py
  📄 translator2.py
  📄 translator3.py
  > llama-models
▼ ui
  > __pycache__
  📄 __init__.py
  📄 controls.py
  📄 ocr_display.py
  📄 translation_display.py
```

DÉVELOPPEMENT

- Technologies utilisées :
 - OCR : EasyOCR, PaddleOCR, Tessseract
 - Traduction : LLaMA local, DeepSeek
 - Hugging face, Ollama
 - Interface : tkinter (PC)
- Étapes principales :
 - Reconnaissance de texte dans l'image
 - Traduction par LLM local
 - Affichage du résultat traduit dans l'interface



Démonstration

BILAN PERSONNEL

- Ce que j'ai appris :
 - Intégration de LLMs locaux pour une tâche ciblée
 - Optimisation des performances (quantization, préchargement, etc.)
 - Utilisation d'outils modernes comme PaddleOCR et ExLLaMA
- Difficultés rencontrées :
 - Compatibilité CUDA/CuDNN
 - Lenteur des premiers chargements
 - OCR de mauvaise qualité en chinois → bascule vers Paddle